Mini-Project Report

Introduction to code

Quicksort is a sorting algorithm that takes on a divide-and-conquer approach. Before it starts sorting, the algorithm must first take in an input array and decide which element to use as a pivot. Typically, this pivot is the first, last, middle, or random element. However, there are also variations of this pivoting strategy, such as the median of three, which takes the first, middle and last elements, sorts them, then chooses the median element to act as the pivot.

After successfully choosing a pivot element, it then begins to partition the array around this chosen pivot such that all elements less than it will be on its left, and all elements greater than it on its right. Upon completing this initial partition, quicksort recursively sorts the left and right subarrays until it reaches a base case where there are either zero or one element left. An important distinction of quicksort is that it does not perform a recombination of all the elements at the end. Instead, at each step of the partitioning, the elements are placed into their correct spots.

A crucial decision that needs to be made when using this algorithm is the selection of the pivot element. A poor pivot choice can potentially lead to uneven partitions and the worst-case time complexity of $O(n^2)$. Conversely, a pivot choice that allows the algorithm to split the array into two roughly equal halves yields optimal performance of O(n log n) time. This mini-project will examine how the performance of the algorithm changes in response to a different pivot element, particularly on how it affects quicksort's speed and efficiency.

Testing Methodology

To thoroughly examine and understand how quicksort behaves, I tested its performance on a variety of inputs and array sizes. Notably, I tested it on sorted, reverse-sorted, and random arrays. Each of these array types was strategically chosen because they demonstrate the algorithms' best-case, worst-case, and average-case performance for partitioning. Furthermore, I also generated a variety of arrays ranging in size from 10000, 49152, and 1000000 elements to examine how cache locality impacts performance. However, I only tested using the last, median, and median of three pivots, since the first element pivot will be similar to the last element pivot in performance and a random pivot would be harder to measure.

To observe the execution time, I used the clock_gettime() C function to measure the time taken. For consistency, I ran the tests multiple times and used the average of the outputs to account for potential interference of background processes. Beyond just timing, I employed the use of the perf tool in Linux to examine my CPU's branch prediction behaviour and branch mispredictions. Perf was particularly useful in that it allowed me to examine which pivoting method suffered the most from horrible branch mispredictions and how caching changed its performance as arrays got larger. Lastly, I used Valgrind's Cachegrind tool to measure cache miss rate and evaluate cache locality.

Analysis of findings

Sorted vs Reverse Sorted vs Random

Quicksort (pivot element)	N=10000	N=49152	N = 1000000
Naive quicksort sorted	3.2 ms	14.9 ms	535.1 ms
Naive quicksort reverse	5.1 ms	25.2 ms	769.3 ms
Naive quicksort random	0.4 ms	2.3 ms	268.9 ms
Median quicksort sorted	0.2 ms	1.2 ms	238.4 ms
Median quicksort reverse	0.2 ms	1.2 ms	252.0 ms
Median quicksort random	0.4 ms	2.2 ms	265.0 ms
Median of three sorted	0.3 ms	4.4 ms	1252.8 ms
Median of three reverse	0.3 ms	2.5 ms	715.7 ms
Median of three random	0.3 ms	2.3 ms	268.4 ms

When the first element or last element was chosen as the pivot, quicksort performed the worst on sorted and reverse-sorted arrays. The pivot being chosen would always either be the largest or smallest element, so the algorithm was unable to divide it evenly and essentially created a one-sided partition. This results in the algorithm performing a larger number of recursive calls and significantly increasing the runtime. When the input was random, the algorithm was allowed to roughly divide the array into two halves.

In contrast, when the pivot chosen was the middle element, it demonstrated consistent and stable performance. This is because the array was able to be partitioned into roughly two even halves, avoiding the worst-case lopsided scenario and maintaining a similar runtime regardless of the type of input array. However, the median of three pivot variation showed some interesting results. While it performed quite similarly to the others on a random array, it was much slower on sorted and reversed-sorted arrays. This is likely due to the extra overhead produced by performing more comparisons, not significantly improving partition balance in already sorted arrays/data.

Array Size and Locality

When the arrays were small or allowed to fit in my L1 cache, the pivoting strategies demonstrated a similar level of performance on random arrays. However, it was quite evident that simply choosing the last or first element as the pivot still resulted in a worse runtime compared to the other implementations because of the aforementioned poor partitioning of the sorted and reverse-sorted arrays. As the arrays grew larger beyond what my cache could hold, the performance difference between the different pivoting strategies became increasingly apparent. The uneven partitions created by some pivots resulted in more recursive calls and

poor cache locality. Each subsequent call using the first or last element as the pivot in sorted/reversed arrays would continuously partition a large chunk of unsorted elements. I found that using the naive pivot results in a L1 cache miss rate of ~1.5%-2% on sorted and reverse-sorted arrays, whereas the random array demonstrated a cache miss rate of 0.4%. The median pivot strategy displayed a consistent miss rate of 0.4% or less across the same-sized input arrays.

Surprisingly, when array sizes became very large, the median of three pivot method performed significantly worse for sorted arrays. While it demonstrated a similar level of performance to the others on a random array, more comparisons and swaps introduced extra overhead that likely contributed to its longer runtime (almost five times slower than its random). However, the cache miss rate on sorted and reverse-sorted arrays was only around 1.0%, with random arrays achieving a cache miss rate of 0.4%.

Branch Behaviour

When arrays were sorted or reverse-sorted, the highly predictable branch nature did not help reduce the poor performance caused by the uneven partitions. Despite the number of branch mispredictions from the comparisons being less than 0.5% and at times basically zero, the algorithm is largely dominated by the uneven partitions. As a result, it ends up making more recursive calls, increasing the number of comparisons and significantly slowing the runtime.

Conversely, random arrays, while much more unpredictable, had much more even partitions. I found the number of branch midpredictions on the comparison to contribute around 36%-40% of the total mispredictions, but because of the significantly reduced number of comparisons, they ran much faster. It seemed that the cost of working on uneven partitions impacted the performance of quicksort more than the benefits of branch predictability.

Conclusions

My findings on quicksort demonstrated that for the algorithm to perform well, the pivot choice plays an integral part in determining its performance. Naively selecting the first or last element, while simple, makes the algorithm extremely susceptible to arrays that are already sorted in either direction. The sorted and reverse-sorted arrays yielded the slowest performance, even though conceptually, it shouldn't be the case. Conversely, a random array, which should yield more branch mispredictions, actually helps the algorithm perform optimally by creating more even partitions and thus fewer recursive calls. While smaller arrays that were able to fit into my processor's L1 cache displayed minimal differences in performance across the different pivoting strategies, larger arrays highlighted the importance of creating even partitions. As these larger arrays went deeper into their recursion, the uneven partitions exacerbated the amount of cache misses, leading to a significant decrease in performance.

So what pivot element should you choose? Based on my results, I would say that the middle element pivot offers the most consistent performance. Choosing a pivot that allows the array to be partitioned into roughly two equal halves will enable the algorithm to minimize recursive calls and reduce the total number of comparisons needed.